CURSO R · MÓDULO 1

Tipos de dados, constantes e variáveis

Aula 2 — 25/09

Ministrantes: Letícia e Jennifer

O que vamos ver hoje

01

Tipos básicos de dados

numeric, character, logical

02

class() e typeof()

como identificar cada tipo

03

Conversão de dados

as.numeric(), as.integer(), as.character()...

04

Constantes e variáveis

atribuição, nomeação, case sensitive

05

Removendo variáveis

ls() e rm()

Por que entender tipos de dados?

Todo programa serve para **ler, processar e disponibilizar dados**.

Para isso, cada função do R exige um tipo (ou classe) de dado específico. Não entender essa “filosofia” dos dados leva a erros comuns no código.

Os 3 tipos básicos que veremos:

numeric | character | logical

```
# exemplo do dia a dia

idade  <- 29          # numeric
nome   <- "Ana"       # character
ativo  <- TRUE         # logical

class(idade)
## [1] "numeric"
```

Três tipos básicos de dados

`numeric`

dados numéricos

inteiros, reais (double) e imaginários (complex)

```
4     4.6     2+3i
```

`character`

dados de texto (string)

sempre entre aspas — senão o R tenta interpretar como comando

```
"Ana Lima"
```

`logical`

dados lógicos

apenas dois valores possíveis, sempre em maiúsculas

```
TRUE     FALSE
```

Dados numéricos: integer, double e complex

class() retorna um tipo mais genérico (“numeric”). **typeof()** é mais detalhado (“double” ou “integer”).

```
# número real -> double
class(4)
## [1] "numeric"
typeof(4)
## [1] "double"

# número inteiro -> use o sufixo L
class(4L)
## [1] "integer"
```

```
# números complexos
class(4 + 3i)
## [1] "complex"

# hierarquia rápida:
# double é a forma "encurtada"
# de double precision,
# alta precisão numérica
```

Dados de texto (character)

Tudo entre aspas é interpretado como string / character — caracteres alfanuméricos.

⚠ **Sem aspas, o R tenta executar como comando:**

```
Ana Lima
## Error: object 'Ana' not found
```

✓ **Com aspas, vira texto normalmente:**

```
"Ana Lima"
## [1] "Ana Lima"
```

```
# números também podem virar texto se estiverem entre aspas
"1234567890"
## [1] "1234567890"

typeof("3")
## [1] "character"
```

Dados lógicos (logical)

Carregam apenas dois valores possíveis. Homenagem ao filósofo/matemático George Boole — por isso “valores booleanos”.

TRUE

FALSE

Sempre em maiúsculas — o R é case sensitive (diferencia maiúsculas de minúsculas).

```
class(TRUE)
## [1] "logical"

# formas abreviadas (primeira letra já é suficiente)
vetor.logico <- c(FALSE, TRUE, F, T)

# lógico -> numérico (coerção)
as.numeric(FALSE)
## [1] 0
```

class() × typeof()

As duas funções identificam o tipo de um dado — mas com granularidades diferentes.

Valor	class()	typeof()
`4`	numeric	double
`4L`	integer	integer
`"3"`	character	character
`TRUE`	logical	logical
`4 + 3i`	complex	complex

class() → tipo generalizado, pouco detalhado | typeof() → tipo detalhado

Conversão de dados

Todas as funções de conversão seguem o prefixo as. — digite-o no console para ver todas as opções.

```
# número real -> inteiro
as.integer(4.6)

# caractere -> numérico
as.numeric("7.8")

# número real -> caractere
as.character(26.8)
```

```
# número complexo -> numérico
as.numeric(2 + 4i)

# lógico -> numérico
as.numeric(FALSE)

# numérico -> lógico
as.logical(1)
```

Nem toda conversão é possível

Quando a conversão não faz sentido, o R gera um aviso (warning) — mas o código não é interrompido.

```
# conversão de texto para inteiro
as.integer("abacate")
## Warning: NAs introduced by coercion
## [1] NA
```

Hierarquia de classes na coerção de vetores:

`logical < integer < numeric < character`

Constantes e variáveis

Atribuir um valor guarda a informação na memória principal do computador, associada a um nome.

`<-`

mais usado, atribui da direita p/ a esquerda

```
n <- 56
```

`->`

atribui da esquerda p/ a direita

```
56 -> n
```

`=`

também funciona para atribuição

```
n = 56
```

`assign()`

atribuição por função

```
assign("n", 56)
```

Variável: informação sujeita a mudanças na memória. **Constante**: informação que não é alterada.

```
# comentários (com #) ajudam a indicar se algo é constante ou variável
IMC_SAUDAVEL <- 22.5   # constante
peso <- 62.5           # variável
```

Regras de nomeação de variáveis

✓ Válido

```
nota3 <- 8
nota.final <- 9
nota_final <- 7
.nota <- 6
._nota <- 5

# pode conter letras, números,
# ponto e underscore
# deve começar com letra ou ponto
```

✗ Inválido

```
2nota <- 8
## não pode começar com número

if <- 5
## "if" é palavra reservada

# mas atenção: o R é case sensitive!
If <- 5       # válido (I maiúsculo)
nota.final != Nota.Final
```

Palavras reservadas (reserved words)

Não podem ser usadas para nomear variáveis ou constantes — exceto se mudar a caixa (ex.: Repeat em vez de repeat).

`if`	`else`	`repeat`	`while`	`function`
`for`	`in`	`next`	`break`	`TRUE`
`FALSE`	`NULL`	`Inf`	`NaN`	`NA`

Removendo variáveis: ls() e rm()

ls() lista todas as variáveis alocadas na memória. **rm()** remove uma ou mais variáveis.

```
# remover uma variável específica
rm(nome do vetor)

# listar todas as variáveis na memória
ls()

# remover TODAS as variáveis e constantes
rm(list = ls())

# conferir que o ambiente ficou vazio
ls()
## character(0)
```

Recapitulando

- 3 tipos básicos: numeric, character, logical
- class() (geral) × typeof() (detalhado)
- Conversão com as.numeric(), as.integer(), as.character()...
- Atribuição: <-, ->, =, assign()
- Nomeação: letras, números, ponto e underscore — sem começar por número
- R é case sensitive · atenção às palavras reservadas
- ls() / rm() para gerenciar a memória

Próximo passo: atividade prática

ATIVIDADE

PREPARAÇÃO

```
v_num <- 15
v_int <- 15L
v_txt <- "15"
v_log <- TRUE
```

1- Qual é a diferença no resultado das funções class() e typeof() para a variável v_num?
2- O que acontece ao realizar a soma de um número inteiro (v_int) com um valor lógico (v_log)?
3- O que ocorre com os tipos dos elementos quando tentamos agrupar número, caractere e valor lógico no mesmo vetor?